Assignment 2: Temperature Data Analysis

Balasathya S (EE24B093)

September 14, 2025

Introduction

This assignment implements a Temperature Data Application Programming Interface (API) using Python. It provides a set of functions to extract, process, and analyze global city temperature data from CSV files.

Two basic functions that form the foundation are:

- **get_city_temperatures:** Gets monthly average temperatures for a given city from the given CSV file, stored in a dictionary.

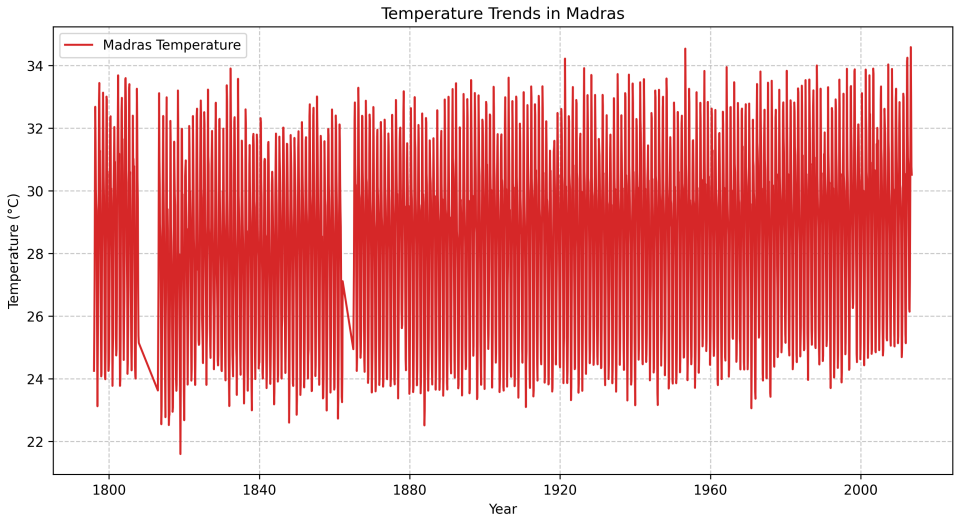


Figure 1: Temperature vs Time(Year) for Madras

- **get_available_cities:** Returns the unique list of cities present in the dataset.

Core API Functions and Logic

The main analytical functions were implemented using the helper routines described earlier. Each function follows a structured approach: extract relevant data, process it, and return results in a clear dictionary format.

1. find_temperature_extremes

This function finds the hottest and coldest months on record for a given city. The implementation retrieves all available temperature records for the city from the .csv file using

`get_city_temperatures`, and then applies Python's in-built `max()` and `min()` functions with the temperature dictionary to locate the respective months. This allows a quick determination of extreme climate events.

2. get_seasonal_averages

This function calculates the average temperature for a given season (spring, summer, fall, or winter). Each season is mapped to its corresponding months, and the function filters the city's monthly records accordingly. The seasonal mean is then obtained by summing the selected monthly temperatures and dividing by the count of values:

$$T_{avg,season} = \frac{\sum_{i=1}^{n} T_i}{n}$$

where T_i represents the temperature in each relevant month. This is particularly useful for understanding seasonal climate behavior.

3. compare_decades

This function compares average temperatures of two decades for a given city. The method involves filtering the dataset by year ranges, computing the average for each, and then taking their difference:

$$\Delta T = T_{decade2} - T_{decade1}$$

If the difference is greater than $0.1°C$, the city is classified as warming; if less than $-0.1°C$, it is classified as cooling; otherwise, the trend is considered stable. This provides a simple measure of long-term temperature change.

4. find_similar_cities

This function finds cities whose average temperatures are close to that of a target city. The average temperature of the target city is computed and compared to all other cities. Cities are selected as similar if the following condition is satisfied:

$$|T_{city} - T_{target}| \leq \tau$$

where: τ is the tolerance threshold specified by the us (the user). This functionality enables comparative climate analysis between cities.

5. get_temperature_trends

This function analyzes long-term climate trends in a city. It groups monthly temperatures into annual averages, computes moving averages across a sliding time window, and applies least squares regression to detect the overall slope of temperature change. The regression slope is computed as:

$$m = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

where: x denotes the years; y the average temperatures; and n the number of years considered. A positive slope indicates a warming trend, while a negative slope indicates cooling. This function provides both fine-grained seasonal analysis and long-term climate perspective.

In the end reformatting was done using 'black' and added minimal comments to the code for readability. And if a city is not found in the csv file, then all the entries return 'DNE'.